

1. O Protagonista (David) - Movimento e Física

```
// --- Inicialização (Create Event) ---

// hsp = 0; vsp = 0; grv = 0.3; walk_spd = 4; jump_spd = -7; hp = 100;
energia_renovavel = 0;


// --- Lógica de Movimento (Step Event) ---

// Captura os inputs do jogador

var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left = keyboard_check(vk_left) || keyboard_check(ord("A"));
var _key_jump = keyboard_check_pressed(vk_space);
var _key_shoot = mouse_check_button_pressed(mb_left);


// Calcula o movimento

var _move = _key_right - _key_left;
hsp = _move * walk_spd;
vsp = vsp + grv;


// Pulo (Lógica Booleana: Se estiver no chão E a tecla for pressionada)
if (place_meeting(x, y + 1, obj_wall)) && (_key_jump) {
    vsp = jump_spd;
}


// Colisão Horizontal
if (place_meeting(x + hsp, y, obj_wall)) {
    while (!place_meeting(x + sign(hsp), y, obj_wall)) {
        x = x + sign(hsp);
    }
    hsp = 0;
}
```

```

}

x = x + hsp;

// Colisão Vertical

if (place_meeting(x, y + vsp, obj_wall)) {
    while (!place_meeting(x, y + sign(vsp), obj_wall)) {
        y = y + sign(vsp);
    }
    vsp = 0;
}

y = y + vsp;

// Animação baseada na folha de sprites de David[cite: 8]

if (!place_meeting(x, y + 1, obj_wall)) {
    sprite_index = spr_david_jump;
} else {
    if (hsp == 0) {
        sprite_index = spr_david_idle;
    } else {
        sprite_index = spr_david_run;
    }
}

if (hsp != 0) image_xscale = sign(hsp); // Vira o sprite para o lado correto

```

2. Combate: Disparo de "Pacotes de Dados"

```

// --- Lógica de Disparo (Adicionar no Step Event do obj_david) ---

if (_key_shoot) {
    // Cria o projétil na posição da arma

```

```
var _inst = instance_create_layer(x, y, "Instances", obj_projectile_data);
```

```
// Define a direção do tiro baseada no rato (Mouse)
```

```
var _dir = point_direction(x, y, mouse_x, mouse_y);
```

```
_inst.direction = _dir;
```

```
_inst.image_angle = _dir; // Rotação do sprite do tiro
```

```
_inst.speed = 10;
```

```
// Altera para a animação de ataque[cite: 8]
```

```
sprite_index = spr_david_shoot;
```

```
}
```

3. Inimigo: Cyber-Zombie (Algoritmo de Busca Básica)

```
// --- Lógica do Inimigo (Step Event de obj_cyber_zombie) ---
```

```
// Estados: 0 = Idle (Esperando conexão), 1 = Chase (Rastreamento de IP), 2 =  
Attack (Injeção de MalWare)
```

```
var _dist_to_player = distance_to_object(obj_david);
```

```
switch (estado) {
```

```
case 0: // Idle
```

```
    sprite_index = spr_zombie_idle; //[cite: 10]
```

```
    hsp = 0;
```

```
    if (_dist_to_player < 200) {
```

```
        estado = 1; // Transição para Chase
```

```
    }
```

```
    break;
```

```
case 1: // Chase
```

```

    sprite_index = spr_zombie_run; //[cite: 10]
    var _dir = sign(obj_david.x - x);
    hsp = _dir * 2; // Velocidade de perseguição

    if (_dist_to_player < 30) {
        estado = 2; // Transição para Attack
    } else if (_dist_to_player > 250) {
        estado = 0; // Perdeu o sinal (Timeout)
    }
    break;

case 2: // Attack

    sprite_index = spr_zombie_attack; //[cite: 10]
    hsp = 0;
    // Lógica de dano aplicada no final da animação
    if (image_index >= image_number - 1) {
        obj_david.hp -= 10;
        estado = 1;
    }
    break;
}

// Aplicar física de gravidade e colisão (semelhante ao jogador)
vsp = vsp + grv;
// ... (inserir lógica de colisão horizontal e vertical aqui)
x += hsp;
y += vsp;

```

4. Puzzles Temáticos: Portões Lógicos (IF / ELSE)

// --- Lógica da Porta de Rede (Step Event de obj_logic_gate) ---

// O portão só abre se (A AND B) for verdadeiro, ou seja, se dois geradores renováveis estiverem ativos.

```
if (global.gerador_solar_ativo == true && global.gerador_eolico_ativo == true) {
```

```
    // Condição lógica satisfeita
```

```
    if (image_alpha > 0) {
```

```
        image_alpha -= 0.05; // Porta cibernética desaparece gradualmente
```

```
    } else {
```

```
        instance_destroy(); // Liberta o caminho para o jogador
```

```
    }
```

```
} else {
```

```
    // Porta trancada - Exibe mensagem "Erro de Sintaxe / Falta de Energia"
```

```
    sprite_index = spr_porta_fechada;
```

```
}
```

5. Interface de Utilizador (HUD) e Sistema de Vidas

// --- Desenhar a HUD (Draw GUI Event) ---

// 1. Desenhar a Vida (Corações)

```
var _espaco = 40;
```

```
for (var i = 0; i < obj_david.hp / 20; i++) {
```

```
    // Desenha um sprite de coração para cada 20 pontos de HP
```

```
    draw_sprite(spr_heart_ui, 0, 20 + (i * _espaco), 20);
```

```
}
```

// 2. Desenhar a Pontuação e Contador de Inimigos

```
draw_set_color(c_white);
```

```
draw_set_font(fnt_retro_terminal); // Assume que tens uma fonte estilo terminal
draw_text(20, 70, "SCORE: " + string(global.pontuacao));
draw_text(20, 90, "ZOMBIES NEUTRALIZADOS: " + string(global.zombies_mortos));
```

```
// 3. Desenhar Status de Energia Renovável (Power Cores)[cite: 7]
```

```
draw_sprite(spr_power_core_small, 0, 20, 120);
draw_text(60, 120, "BATERIAS: " + string(global.energia_renovavel) + " / 3");
```

6. Coletáveis: Núcleos de Energia Renovável (Power Cores)

```
// --- Evento de Colisão com obj_david ---
```

```
global.energia_renovavel += 1;
```

```
global.pontuacao += 50;
```

```
// Efeito sonoro de "Download de Dados / Carga Completa"
```

```
audio_play_sound(snd_power_up, 1, false);
```

```
// Partículas de energia limpa (verde)
```

```
effect_create_above(ef_flare, x, y, 1, c_lime);
```

```
// Destrói a bateria no mapa
```

```
instance_destroy();
```

7. Boss Fight: A Aranha JavaScript/Script Injector

Create Event:

```
hp_boss = 500;
```

```
estado_boss = "IDLE"; // Estados: IDLE, INJECT_SCRIPT (Invocar), VULNERAVEL
```

```
timer_ataque = 120; // Tempo entre ataques
```

Step Event (Máquina de Estados do Boss):

```
timer_ataque -= 1;
```

```

if (timer_ataque <= 0) {
    if (estado_boss == "IDLE") {
        estado_boss = "INJECT_SCRIPT";
        timer_ataque = 60; // Duração do ataque
    } else if (estado_boss == "INJECT_SCRIPT") {
        estado_boss = "VULNERAVEL";
        timer_ataque = 180; // Tempo que o olho/núcleo fica exposto
    } else {
        estado_boss = "IDLE";
        timer_ataque = 120;
    }
}

// Lógica de cada estado
switch(estado_boss) {
    case "INJECT_SCRIPT":
        // O Boss injeta o "Loop Infinito Cloner Zombie"
        // Se houver menos de 5 zombies na sala, cria mais um
        if (instance_number(obj_cyber_zombie) < 5) {
            instance_create_layer(x, y + 50, "Enemies", obj_cyber_zombie);
        }
        sprite_index = spr_aranha_atacando;
        break;

    case "VULNERAVEL":
        // O núcleo central azul brilhante abre-se e recebe dano duplo
        sprite_index = spr_aranha_vulneravel;
        break;
}

```

```
case "IDLE":  
    sprite_index = spr_aranha_base; // A forma gigante de zombies[cite: 3]  
    break;  
}
```

```
// Lógica de Morte  
  
if (hp_boss <= 0) {  
    // Executa a animação de destruição do servidor  
    instance_destroy();  
}
```

8. Lógica Baseada em Matemática: O Chefe Vértice

Create Event (do projétil):

```
tempo_x = 0; // O nosso 'x' da equação  
posicao_inicial_y = y;  
posicao_inicial_x = x;  
velocidade_x = 3 * sign(obj_david.x - x); // Direção para o jogador
```

Step Event (do projétil):

```
// Incrementamos o eixo X virtual  
tempo_x += 0.1;
```

```
// Movemos o eixo X real  
x += velocidade_x;
```

```
// Aplicamos a equação do Chefe Vértice:  $y = -x^2 + 4x$ [cite: 2]  
// Multiplicamos por um fator (ex: 10) para escalar visualmente no jogo  
var_calculo_y = -(power(tempo_x, 2)) + (4 * tempo_x);
```

```
// Atualizamos a posição Y baseada na equação (subtraímos porque no  
GameMaker o Y inverte para baixo)
```

```
y = posicao_inicial_y - (_calculado_y * 10);
```

```
// Destruir ao atingir o chão
```

```
if (place_meeting(x, y, obj_wall)) {
```

```
    instance_destroy();
```

```
}
```

9. Mecânica de Detecção: O Sequestrador de IP

Create Event:

```
alcance_visao = 300;
```

```
alerta = 0; // Vai de 0 a 100
```

```
estado = "PROCURANDO";
```

Step Event:

```
// Verifica se há uma linha limpa (sem paredes) entre o inimigo e o David
```

```
var _visivel = !collision_line(x, y, obj_david.x, obj_david.y, obj_wall, false, true);
```

```
var _distancia = distance_to_object(obj_david);
```

```
if (_distancia < alcance_visao && _visivel) {
```

```
    // Jogador detetado: aumenta o nível de alerta
```

```
    alerta += 2;
```

```
    sprite_index = spr_sequestrador_alertando; // Animação de transição
```

```
if (alerta >= 100 && estado != "DETECTADO") {
```

```
    estado = "DETECTADO";
```

```
    // Cria a notificação visual "IP verdadeiro detectado!"[cite: 2]
```

```
    instance_create_layer(x, y - 40, "UI", obj_aviso_ip);
```

```

// Atrai os Cyber-Zombies mais próximos alterando o estado deles
with (obj_cyber_zombie) {
    if (distance_to_object(other) < 400) {
        estado = 1; // Força-os para o estado de perseguição (Chase)
    }
}
}
} else {
    // Jogador escondido: diminui o alerta
    if (alerta > 0) alerta -= 1;
    if (alerta == 0) {
        estado = "PROCURANDO";
        sprite_index = spr_sequestrador_base; // Volta aos tentáculos escuros[cite: 2]
    }
}
}

```

10. Puzzle Avançado: O Terminal IF / ELSE IF / ELSE

Step Event (Interação com o Terminal):

```

var _perto = distance_to_object(obj_david) < 30;
var _tecla_interacao = keyboard_check_pressed(ord("E"));

if (_perto && _tecla_interacao) {
    // Avalia a quantidade de energia renovável armazenada na variável global
    if (global.energia_renovavel >= 3) {
        // Bloco IF: Desbloqueia a rota segura e consome 3 energias
        obj_porta_A.aberta = true;
        global.energia_renovavel -= 3;
        audio_play_sound(snd_acesso_permitido, 1, false);
    }
}

```

```

} else if (global.energia_renovavel == 2) {

    // Bloco ELSE IF: Rota alternativa, consome 2 energias

    obj_porta_B.aberta = true;

    global.energia_renovavel -= 2;

    audio_play_sound(snd_acesso_parcial, 1, false);

} else {

    // Bloco ELSE ("boa sorte"[cite: 2]): Sem energia suficiente, abre o poço dos
monstros

    obj_porta_C.aberta = true;

    audio_play_sound(snd_erro_critico, 1, false);

}

}

```

11. O Portal do Loop (Conclusão do Nível)

Create Event:

```

ativo = false;

energia_necessaria = 5; // Requer 5 núcleos pequenos para ativar

```

Step Event:

```

// Ativação do Portal

if (!ativo && global.energia_renovavel >= energia_necessaria) {

    // Verifica se o jogador pressionou a tecla de interação no portal

    if (place_meeting(x, y, obj_david) && keyboard_check_pressed(ord("W"))) {

        ativo = true;

        global.energia_renovavel -= energia_necessaria;

        // Aplica o efeito visual PORTAL_FX por cima do PORTAL_GATE[cite: 7]

        var _fx = instance_create_layer(x, y, "Efeitos", obj_portal_fx);

        _fx.image_speed = 1.5;
    }
}

```

```

        // Inicia o alarme para mudar de sala (Room) após 2 segundos

        alarm[0] = room_speed * 2;

    }

}

```

Alarm 0 Event (Transição de Room):

// Transporta o David para o próximo nível (ex: Da Zona Urbana para a Instalação Científica)

```

room_goto_next();

```

12. Plataformas Dinâmicas: A Passadeira Rolante (Loop Infinito)

Create Event (da passadeira):

velocidade_esteira = -3; // Valor negativo empurra para a esquerda, positivo para a direita

Step Event (adicionar ao código do obj_david):

// Verifica se o jogador tem uma passadeira imediatamente abaixo dos pés

```

var _esteira_abaixo = instance_place(x, y + 1, obj_passadeira);

```

```

if (_esteira_abaixo != noone) {

```

```

    // Adiciona a velocidade da esteira ao vetor X do jogador

```

```

    x += _esteira_abaixo.velocidade_esteira;

```

// Força a animação de corrida se o jogador estiver a tentar ficar parado contra a passadeira

```

    if (hsp == 0) {

```

```

        sprite_index = spr_david_run; //[cite: 8]

```

```

    }

```

```

}

```

13. Batalha Épica: O Gigante Cyber-Zombie (Boss Modular)

Create Event (Montagem do Boss):

```
hp_total = 1500;
```

```
fase = 1;
```

```
// Instancia as partes na tela usando coordenadas relativas ao  
controlador[cite: 7]
```

```
torso = instance_create_layer(x, y, "Boss_Layer", obj_boss_torso);
```

```
cabeca = instance_create_layer(x, y - 100, "Boss_Layer", obj_boss_cabeca);
```

```
// Os braços precisam de lógica independente de ataque
```

```
braco_esq = instance_create_layer(x - 90, y - 20, "Boss_Layer",  
obj_boss_braco);
```

```
braco_dir = instance_create_layer(x + 90, y - 20, "Boss_Layer",  
obj_boss_braco);
```

```
braco_esq.image_xscale = -1; // Inverte o sprite para o braço esquerdo
```

Step Event (Comportamento Sincronizado):

```
// Transfere o dano das partes vulneráveis (ex: Cabeça e Torso com carne  
exposta[cite: 7]) para o HP total
```

```
if (cabeca.tomou_dano) {
```

```
    hp_total -= 20;
```

```
    cabeca.tomou_dano = false; // Reseta a flag de dano
```

```
}
```

```
// Coordena os ataques na Fase 1
```

```
if (fase == 1) {
```

```
    // 2% de probabilidade por frame de lançar um ataque esmagador com um  
    dos braços
```

```
    if (irandom(100) > 98 && braco_esq.estado == "IDLE") {
```

```

        braco_esq.estado = "ATACAR_ESMAGAR";
    }
}

// Sequência de Destruição
if (hp_total <= 0) {
    instance_create_layer(x, y, "Efeitos", obj_explosao_massiva);
    instance_destroy(torso);
    instance_destroy(cabeca);
    instance_destroy(braco_esq);
    instance_destroy(braco_dir);
    instance_destroy(); // Destrói o controlador
}

```

14. Efeitos de Morte: Limpeza de Memória (Glitch Skull)

Destroy Event (do obj_cyber_zombie):

```

// Atualiza a HUD[cite: 7]
global.zombies_mortos += 1;
global.pontuacao += 150;

// Cria o cadáver que fará a animação
var _corpo = instance_create_layer(x, y, "Corpos", obj_zombie_morto);
_corpo.sprite_index = spr_zombie_death; //[cite: 10]
_corpo.image_xscale = image_xscale; // Garante que cai para o lado certo

// 30% de probabilidade de largar energia renovável (Drop do inimigo)

```

```

if (irandom(100) <= 30) {
    instance_create_layer(x, y - 10, "Coletaveis", obj_power_core);
}

```

Animation End Event (do obj_zombie_morto):

```

if (sprite_index == spr_zombie_death) {
    // Quando a animação de queda termina, muda para o sprite de holograma
    glitchado[cite: 10]

    sprite_index = spr_glitch_skull;

    alarm[0] = room_speed * 1.5; // Inicia um temporizador para desaparecer em 1.5
    segundos
}

```

15. O Final: O Protocolo de Resgate (Byte-Earth)

```

draw_set_halign(fa_center);
draw_set_font(fnt_retro_terminal);

// Mensagem de Sucesso do Protocolo
draw_set_color(c_white);

draw_text(display_get_gui_width() / 2, display_get_gui_height() / 2 - 50, "SISTEMA
TERRESTRE REINICIADO");

draw_text(display_get_gui_width() / 2, display_get_gui_height() / 2, "PROTOCOLO
DE RESGATE CONCLUIDO");

// Apresentação dos resultados lidos das variáveis globais
draw_set_color(c_lime);

draw_text(display_get_gui_width() / 2, display_get_gui_height() / 2 + 60, "SCORE
FINAL: " + string(global.pontuacao));

draw_text(display_get_gui_width() / 2, display_get_gui_height() / 2 + 90, "AMEACAS
NEUTRALIZADAS: " + string(global.zombies_mortos));

```

1. Sistema do David e Gestão de RAM (Subsolo do Metrô)

```

// --- obj_david (Evento Create) ---

hp = 100;

velocidade_base = 5;

velocidade_atual = 5;

carga_ram_atual = 0;

carga_ram_maxima = 1024; // Medido em Megabytes simulados

escudo_ativo = false;

mostrar_alerta_hud = "";


// --- obj_david (Evento Step) ---

var _move = keyboard_check(vk_right) - keyboard_check(vk_left);


// Calcula a penalização de velocidade baseada na RAM ocupada
var _penalizacao_ram = (carga_ram_atual / carga_ram_maxima) * 3;
velocidade_atual = max(1, velocidade_base - _penalizacao_ram);


x += _move * velocidade_atual;


// Alerta de RAM crítica e otimização manual por descarte (Tecla Q)
if (carga_ram_atual > (carga_ram_maxima * 0.75)) {
    mostrar_alerta_hud = "RAM CRÍTICA: OTIMIZE ARMAZENAMENTO";
    if (keyboard_check_pressed(ord("Q"))) {
        carga_ram_atual = max(0, carga_ram_atual - 256); // Descarta pacotes
        supérfluos[cite: 12]
        mostrar_alerta_hud = "";
    }
}

```

2. Puzzles Condicionais: O Portão Lógico

```

// --- obj_totem_clima (Evento de Interação) ---

// O jogador escolhe o totem marcado com "Chuva de Ácido == true"[cite: 12]
if (condicao_totem == "Chuva de Ácido") {
    global.chuva_acido = true;

    // Ativa o escudo hexagonal azul em redor de David[cite: 12]
    obj_david.escudo_ativo = true;

    instance_create_layer(obj_david.x, obj_david.y, "Efeitos",
obj_escudo_hexagonal);

    // O portão lógico processa a condição e abre-se
    with (obj_portao_logico) {
        estado = "ABERTO";

        // O outdoor exibe a mensagem de ironia[cite: 12]
        obj_outdoor.texto = "else { boa sorte }";
    }
}

```

3. Boss Fight: O Zumbi Gigante da Esteira (Loop Infinito)

```

// --- obj_boss_esteira (Evento Step) ---

if (loopCondition == true) {
    // A esteira gira em velocidade máxima[cite: 12]
    velocidade_esteira = 10;

    // Gera clones continuamente[cite: 12]
    if (alarm[0] < 0) {
        instance_create_layer(x, y - 50, "Inimigos", obj_clone_zumbi);
    }
}

```

```

        alarm[0] = room_speed * 2;

    }

} else {

    // Quando David atinge a caixa de energia com o comando 'break'[cite: 12]

    velocidade_esteira = lerp(velocidade_esteira, 0, 0.05); // Desacelera até
    travar[cite: 12]

    sprite_index = spr_boss_engasgado;

}

// --- obj_caixa_energia (Evento de Colisão com obj_tiro_depuracao) ---

// Injeta um comando 'break' na variável de controlo[cite: 12]

obj_boss_esteira.loopCondition = false;

instance_destroy();

```

4. Boss Fight: O Sequestrador de IP e Máscaras de Sub-Rede

```

// --- obj_sequestrador_ip (Evento Step) ---

// O Boss dispara pacotes vermelhos (DDoS)

if (irandom(100) < 5) {

    var_ddos = instance_create_layer(x, y, "Ataques", obj_pacote_ddos);

    _ddos.image_blend = c_red;

    _ddos.direction = point_direction(x, y, obj_david.x, obj_david.y);

}

// --- obj_servidor_ip (Evento de Colisão com obj_tiro_depuracao) ---

// David deduz que a Máscara 255.255.255.192 aponta para este servidor
específico[cite: 13]

if (ip_valido == true) {

    // O disfarce do monstro cai, revelando o núcleo[cite: 13]

    with (obj_sequestrador_ip) {

```

```

    estado = "VULNERAVEL";

    mostrar_mensagem = "IP verdadeiro detectado!"; //[cite: 13]

    alarm[1] = room_speed * 1; // Intervalo minúsculo para David atacar[cite: 13]
}
}

```

5. Manipulação Urbana de HTML/CSS e a Aranha Script Injector

```

// --- obj_aranha_css (Evento Alarm - Ataque de Rotação) ---

// A aranha invoca propriedades CSS no mundo[cite: 13]
global.mundo_rotacao = 180; // Inverte o cenário e a gravidade
global.mundo_cor_fundo = "#000"; // Escurece o ecrã[cite: 13]
global.mundo_opacidade = 0.1; //[cite: 13]

// --- obj_pilar_arena (Evento de Colisão com obj_tiro_depuracao) ---

// David reescreve as classes disparando nos pilares[cite: 13]
if (classe_alvo == "gravidade") {
    global.mundo_rotacao = 0; // "classe.gravidade { girar: 0deg; }"[cite: 13]
} else if (classe_alvo == "luz") {
    global.mundo_opacidade = 1.0; // "classe.luz { visibilidade: visível; }"[cite: 13]
}

// Quando ambas as classes são corrigidas, a aranha cai vulnerável no centro[cite: 13]
if (global.mundo_rotacao == 0 && global.mundo_opacidade == 1.0) {
    obj_aranha_css.estado = "QUEDA_PESADA";
}

```

6. Instalação Científica: Funções Matemáticas e O Vértice

```

// --- obj_boss_vertice (Evento Step) ---

// O Boss move-se segundo a equação matemática[cite: 14]

```

```

x += velocidade_x;

var _tempo_x = x / 50; // Escala do gráfico

y = posicao_inicial_y - ((-power(_tempo_x, 2)) + (4 * _tempo_x)) * 50;

// --- obj_armadilha_energia (Evento Step) ---

// David calcula mentalmente o vértice usando  $-b/2a$  para posicionar a
armadilha[cite: 14]

// Para  $y = -x^2 + 4x$ ,  $a = -1$ ,  $b = 4$ . Vértice  $x = -4 / (2*-1) = 2$ .

var _posicao_vertice_x = 2 * 50;

if (abs(obj_boss_vertice.x - _posicao_vertice_x) < 10) {
    // A armadilha explode exatamente no ponto luminoso (pico da curva)[cite: 14]
    instance_create_layer(x, y, "Efeitos", obj_explosao_branca);
    obj_boss_vertice.estado = "QUEDA_LIVRE"; // Destrói a blindagem[cite: 14]
}

```

7. O Confronto Final: Paul e o Núcleo do Servidor

```

// --- obj_boss_paul (Evento Step) ---

// Criação de armadilhas HTML

if (irandom(100) < 2) {
    instance_create_layer(x_aleatorio, chão_y, "Armadilhas", obj_falso_buraco_div);
}

// --- obj_david (Disparo de Depuração Máxima contra Paul) ---

// Ao atingir Paul, injeta comentários e estruturas try/catch[cite: 15]

with (obj_boss_paul) {
    hp_armadura -= dano_depuracao;

    if (hp_armadura <= 0) {
        // Exceção fatal: faíscas e quebra da armadura[cite: 15]
    }
}

```

```

    estado = "ARMADURA_ROMPIDA";

    instance_create_layer(x, y, "Efeitos", obj_codigo_vermelho_erro);

}

}

```

8. O Terminal de Escolha Final

```

// --- obj_painel_final (Evento Draw GUI) ---

// Desenha os botões holográficos determinantes para o destino da Terra[cite: 15]

draw_set_color(c_blue);

draw_text(x_esq, y_centro, "Inject Cure();"); // Botão azul de David[cite: 15]


draw_set_color(c_red);

draw_text(x_dir, y_centro, "Consolidate Control();"); // Botão vermelho de Paul[cite: 15]


// Lógica de interação

if (mouse_check_button_pressed(mb_left)) {

    if (rato_sobre_botao_azul) {

        global.final = "CURA";

        room_goto(rm_cinematica_cura);

    } else if (rato_sobre_botao_vermelho) {

        global.final = "CONTROLO";

        room_goto(rm_cinematica_dominio);

    }

}

```

Fase 3: Subsolo do Metrô (Gestão de RAM e Protocolos de Rede)

Mecânica 1: Otimização de RAM e a Varredura Automática (Garbage Collector)

```

// --- obj_rotina_limpeza (Evento Step - A parede que persegue David) ---

```

```

// A rotina avança constantemente
x += velocidade_varredura;

if (place_meeting(x, y, obj_david)) {
    // Game Over se a varredura automática apanhar o jogador[cite: 12]
    room_restart();
}

// --- obj_david (Evento Step - Atualização do peso da RAM) ---
var _penalizacao = (carga_ram / max_ram) * 4;
velocidade_atual = 5 - _penalizacao;

if (velocidade_atual <= 2) {
    // Exibe o alerta no HUD projetado na viseira[cite: 12]
    mostrar_alerta_hud = "RAM CRÍTICA: OTIMIZE ARMAZENAMENTO";
}

// Descartar itens para recuperar velocidade[cite: 12]
if (keyboard_check_pressed(ord("Q")) && carga_ram > 0) {
    carga_ram -= 256; // Descarta pacotes supérfluos[cite: 12]
}

```

Mecânica 2: Ponte de Protocolos de Rede

```

// --- obj_ponte_dados (Evento Create) ---
ordem_correta = ["Física", "Enlace", "Rede"]; //[cite: 12]
pacotes_inseridos = [];
ponte_solida = false;
sprite_index = spr_bloco_semitransparente; //[cite: 12]

```

```
// --- obj_ponte_dados (Evento de Recebimento de Pacote via Drag & Drop) ---
// Quando David arrasta um pacote holográfico para a ponte[cite: 12]
array_push(pacotes_inseridos, pacote_arrastado.tipo);

var _index = array_length(pacotes_inseridos) - 1;

if (pacotes_inseridos[_index] != ordem_correta[_index]) {
    // Erro de protocolo: a ponte treme e desfaz-se em partículas[cite: 12]
    effect_create_above(ef_spark, x, y, 1, c_white);
    pacotes_inseridos = []; // Reseta o puzzle
    audio_play_sound(snd_curto_circuito, 1, false); //[cite: 12]
} else if (array_length(pacotes_inseridos) == 3) {
    // Ordem correta inserida: solidifica a ponte[cite: 12]
    ponte_solida = true;
    sprite_index = spr_bloco_solido;
}
}
```

Fase 5: Zona Urbana HTML/CSS

Mecânica 1: Desativar Lasers com visibility: hidden;

```
// --- obj_laser_css (Evento de Colisão com obj_tiro_depuracao) ---
if (atributo_visibilidade == "visible") {
    // Redefine o atributo com um tiro preciso
    atributo_visibilidade = "hidden";

    // O laser resume-se a uma linha de estilo comentada, perdendo a colisão[cite:
13]
    sprite_index = spr_laser_comentado;
    mask_index = spr_vazio; // Remove a hitbox letal
}
```

```
}
```

Mecânica 2: Ajuste de Passarelas com width e height

```
// --- obj_painel_slider (Evento Step - Arrastar o cursor com o rato) ---  
  
if (mouse_check_button(mb_left) && position_meeting(mouse_x, mouse_y,  
obj_cursor_slider)) {  
  
    // Arrasta o cursor horizontalmente[cite: 13]  
  
    obj_cursor_slider.x = clamp(mouse_x, slider_min_x, slider_max_x);  
  
  
    // Calcula a percentagem da largura e aplica à plataforma correspondente  
  
    var _percentagem = (obj_cursor_slider.x - slider_min_x) / (slider_max_x -  
slider_min_x);  
  
    plataforma_alvo.image_xscale = 1 + (_percentagem * 4); // Aumenta a largura  
(width)[cite: 13]  
  
}
```

Fase 6: Instalação Científica (Física de Funções)

Inimigo Base: Zumbis de Código em Trajetória "U"

```
// --- obj_zumbi_matematico (Evento Create) ---  
  
// Define a equação para a parábola em forma de U  
  
x_inicial = x;  
y_inicial = y;  
  
param_a = 0.5; // Curva voltada para cima  
param_b = -2;  
param_c = 0;  
  
tempo = -5; // Começa a desenhar do lado esquerdo da curva  
  
  
  
  
  
  
  
  
  
// --- obj_zumbi_matematico (Evento Step) ---  
  
tempo += 0.1; // Avança no eixo X virtual
```

```
// Calcula o eixo Y real do inimigo baseado na equação
var _novo_y = (param_a * power(tempo, 2)) + (param_b * tempo) + param_c;

x = x_inicial + (tempo * 15);
y = y_inicial + (_novo_y * 15);

// Desenha a equação em 3D holográfico perto do inimigo (no Draw Event)
```

Mecânica: Plataformas Paramétricas

```
// --- obj_controlador_funcao (Evento Step) ---
// Quando David ajusta os sliders (ex: altera a variável global.param_a)[cite: 14]
if (global.param_a_alterado) {
  with (obj_plataforma_parabola) {
    // "Se eu aumentar o 'a', a parábola fica mais fechada"[cite: 14]
    // A plataforma move-se para o novo vértice calculado
    novo_vertice_x = -global.param_b / (2 * global.param_a);
    novo_vertice_y = (global.param_a * power(novo_vertice_x, 2)) +
(global.param_b * novo_vertice_x) + global.param_c;

    // Move suavemente para a nova coordenada
    x = lerp(x, x_base + novo_vertice_x, 0.1);
    y = lerp(y, y_base + novo_vertice_y, 0.1);
  }
}
```

Fase 7: Torre de Transmissão (Lógica e Roteamento)

Mecânica: Painel de Roteamento de Elevadores

```
// --- obj_painel_roteamento (Evento Step) ---  
  
// O jogador arrasta o ícone do pacote pelos caminhos de roteadores na  
interface[cite: 14]  
  
if (pacote_chegou_ao_destino) {  
    // Ativa o elevador na torre[cite: 14]  
  
    with (obj_elevador_torre) {  
        ativo = true;  
        velocidade_y = -4; // Sobe para o próximo nível[cite: 14]  
    }  
  
    // Hordas de zumbis de código escalam a estrutura logo a seguir[cite: 14]  
  
    instance_create_layer(x_torre_esq, y_base, "Inimigos", obj_zumbi_avancado);  
    instance_create_layer(x_torre_dir, y_base, "Inimigos", obj_zumbi_avancado);  
}
```

```
// Portas Blindadas (Equações de 1º Grau)  
  
// Exigem respostas a equações lineares para liberar passagem[cite: 14]  
  
if (resposta_inserida_console == solucao_equacao_1_grau) {  
    obj_porta_blindada.y -= 100; // Abre a porta[cite: 14]  
}
```

1. Boss: O Zumbi Gigante da Esteira (Loop Infinito)

```
// --- obj_boss_esteira (Evento Create) ---  
  
// O zumbi repete mecanicamente: "while(true) { gerar_clone(); }"[cite: 12]  
  
loopCondition = true;  
  
timer_clonagem = room_speed * 3;  
  
velocidade_animacao = 1.5;
```

```
// --- obj_boss_esteira (Evento Step) ---
if (loopCondition == true) {
    timer_clonagem -= 1;
    if (timer_clonagem <= 0) {
        // Clones menores saltam da esteira e avançam sobre David[cite: 12]
        instance_create_layer(x, y + 64, "Inimigos", obj_clone_zumbi);
        timer_clonagem = room_speed * 3;
    }
} else {
    // A esteira engasga, chia e desacelera até travar num silêncio com fumaça
    pixelizada[cite: 12]
    velocidade_animacao = lerp(velocidade_animacao, 0, 0.02);
    image_speed = velocidade_animacao;
    if (image_speed < 0.1) image_speed = 0;
}
```

```
// --- obj_caixa_energia (Evento de Colisão com obj_tiro_depuracao) ---
// Um painel exibe a linha `loopCondition = true;` [cite: 12]
// O tiro de energia azul insere um comando 'break'[cite: 12]
obj_boss_esteira.loopCondition = false;
audio_play_sound(snd_esteira_engasgando, 1, false);
instance_destroy();
```

2. Boss: O Sequestrador de IP

```
// --- obj_sequestrador_ip (Evento Step) ---
// Dispara um enxame de pacotes vermelhos representando ataques DDoS como
mísseis
if (estado == "ATACANDO" && irandom(100) < 5) {
    var_ddos = instance_create_layer(x, y, "Projeteis", obj_pacote_ddos);
```

```

_ddos.image_blend = c_red;

_ddos.direction = point_direction(x, y, obj_david.x, obj_david.y);

_ddos.speed = 8;
}

// --- obj_servidor_ip (Evento de Colisão com obj_tiro_depuracao) ---

// O jogador analisa: "Máscara 255.255.255.192... então a sub-rede válida é...
aquele servidor!"[cite: 13]

if (ip_correto == true) {
    with (obj_sequestrador_ip) {
        // O disfarce do monstro desfaz-se, exibindo o seu núcleo brilhante[cite: 13]
        estado = "VULNERAVEL";
        mensagem_sistema = "IP verdadeiro detectado!"; //[cite: 13]

        // Intervalo minúsculo para David lançar um disparo concentrado[cite: 13]
        alarm[0] = room_speed * 1.5;
    }
} else {
    // Se atirar na isca, o boss teleporta-se e ataca mais rápido
    obj_sequestrador_ip.x = choose(posicao_esq, posicao_dir);
}

```

3. Boss: A Aranha Script Injector

```

// --- obj_aranha_css (Evento Alarm - Injeção de Código) ---

// Envia o comando 'transform: rotate(180deg)' que faz o mundo girar de cabeça
para baixo[cite: 13]

global.mundo_rotacao = 180;

// Envia comando 'background-color: #000;' e 'opacity: 0.1;' mergulhando a arena
em quase total escuridão[cite: 13]

```

```

global.mundo_iluminacao = 0.1;

// --- obj_pilar_arena (Evento de Colisão com obj_tiro_depuracao) ---
// David reescreve as classes com balas de código[cite: 13]
if (classe_alvo == "gravidade") {
    // "classe.gravidade { girar: 0deg; }"[cite: 13]
    global.mundo_rotacao = 0;
    sprite_index = spr_pilar_restaurado;
} else if (classe_alvo == "luz") {
    // "classe.luz { visibilidade: visivel; }"[cite: 13]
    global.mundo_iluminacao = 1.0;
    sprite_index = spr_pilar_restaurado;
}

// Privada do controle de estilo, a aranha cai pesada no centro[cite: 13]
if (global.mundo_rotacao == 0 && global.mundo_iluminacao == 1.0) {
    with (obj_aranha_css) {
        estado = "VULNERAVEL";
        y = y_centro_chao; // Cai pesada[cite: 13]
    }
}

```

4. Boss: O Chefe Vértice

```

// --- obj_chefe_vertice (Evento Step) ---
// Move-se deixando rastros da fórmula:  $y = -x^2 + 4x$ [cite: 14]
x += velocidade_x;
var _x_grafico = (x - x_centro_sala) / escala_grafica;

// Calcula a trajetória perfeita de segundo grau[cite: 14]

```

```

var _y_grafico = (-power(_x_grafico, 2)) + (4 * _x_grafico);
y = y_base_chao - (_y_grafico * escala_grafica);

// --- obj_armadilha_energia (Evento Step) ---
// David posiciona a armadilha onde o vértice será atingido ( $x = -b/2a = 2$ )[cite: 14]
var _posicao_vertice_x = x_centro_sala + (2 * escala_grafica);
var _posicao_vertice_y = y_base_chao - (4 * escala_grafica);

// Se a armadilha explodir exatamente no ponto luminoso calculado quando o
chefe passa[cite: 14]
if (abs(obj_chefe_vertice.x - _posicao_vertice_x) < 10 && abs(obj_chefe_vertice.y -
_posicao_vertice_y) < 10) {
    // Explosão em clarão branco, destruindo a blindagem e forçando queda
livre[cite: 14]
    instance_create_layer(x, y, "Efeitos", obj_clarao_branco);
    with (obj_chefe_vertice) {
        estado = "QUEDA_LIVRE";
        velocidade_x = 0;
        gravity = 0.5; // Os números desfazem-se como giz[cite: 14]
    }
}

```

5. Boss Final: Paul e o Núcleo do Servidor Central

```

// --- obj_boss_paul (Evento Step) ---
if (estado == "COMBATE") {
    timer_habilidade -= 1;

    if (timer_habilidade <= 0) {
        var _acao = choose("DIV_FALSO", "CAMPO_WHILE", "VETOR_MATEMATICO");
    }
}

```

```

switch(_acao) {
    case "DIV_FALSO":
        // Distorce o cenário abrindo buracos falsos com tags <div> que surgem e
        desaparecem

        instance_create_layer(x_aleatorio_chao, y_chao, "Armadilhas",
obj_buraco_div);

        break;

    case "CAMPO_WHILE":
        // Prende David em campos de energia que repetem comandos
        'while(true)' ao seu redor[cite: 15]

        instance_create_layer(obj_david.x, obj_david.y, "Armadilhas",
obj_campo_while);

        break;

    case "VETOR_MATEMATICO":
        // Lança vetores matemáticos em forma de projéteis curvos[cite: 15]

        var _vetor = instance_create_layer(x, y, "Projeteis", obj_projetil_vetor);

        _vetor.direction = point_direction(x, y, obj_david.x, obj_david.y);

        break;

}

timer_habilidade = room_speed * 3;

}
}

```

6. A Decisão Final (Sistema de Múltiplos Finais)

```

// --- obj_painel_final (Evento Draw GUI) ---

draw_set_halign(fa_center);

draw_set_font(fnt_terminal_gigante);


// A tela central exibe a enorme condicional do sistema[cite: 15]

draw_set_color(c_white);

```

```
draw_text(display_get_gui_width() / 2, 100, "if (escolha == aliar_a_paul) {?} else  
{?}"); //[cite: 15]
```

```
// Desenha o botão azul "Inject Cure()";[cite: 15]
```

```
draw_set_color(c_blue);
```

```
draw_text(display_get_gui_width() / 3, display_get_gui_height() / 2, "Inject Cure()");  
//[cite: 15]
```

```
// Desenha o botão vermelho "Consolidate Control()";[cite: 15]
```

```
draw_set_color(c_red);
```

```
draw_text((display_get_gui_width() / 3) * 2, display_get_gui_height() / 2,  
"Consolidate Control()"); //[cite: 15]
```

```
// --- obj_painel_final (Evento Step - Lógica de Seleção) ---
```

```
// Verifica o clique do jogador para executar a rotina de encerramento do jogo
```

```
if (mouse_check_button_pressed(mb_left)) {
```

```
    if (rato_em_cima_do_azul) {
```

```
        // David escolhe o caos e a liberdade, limpando a rede da corrupção  
matemática de Paul[cite: 15]
```

```
        global.final_escolhido = "CURA";
```

```
        room_goto(rm_cinematica_final_azul);
```

```
    } else if (rato_em_cima_do_vermelho) {
```

```
        // David junta-se a Paul para governar a rede com eficiência matemática  
perfeita[cite: 15]
```

```
        global.final_escolhido = "CONTROLE";
```

```
        room_goto(rm_cinematica_final_vermelho);
```

```
    }
```

```
}
```

Inimigos Menores

Clones do Zumbi Gigante (Praça)

```

// --- obj_clone_zumbi (Evento Create) ---

hp = 20;

velocidade = 2; // Movimento trôpego, mas constante

dano = 10;

audio_play_sound(snd_ranger_metal, 1, true); // Som de rangido de metal[cite: 12]


// --- obj_clone_zumbi (Evento Step) ---

// Avançam incansavelmente na direção de David[cite: 12]

var _dir = sign(obj_david.x - x);

x += _dir * velocidade;


// Colisão com o jogador

if (place_meeting(x, y, obj_david) && obj_david.escudo_ativo == false) {

    obj_david.hp -= dano;

    instance_destroy();

}

```

Hordas de Zumbis de Código Avançados (Torre de Transmissão)

```

// --- obj_zumbi_avancado (Evento Create) ---

hp = 40;

velocidade_escalada = 3;

timer_piscar_olhos = 10;


// --- obj_zumbi_avancado (Evento Step) ---

// Mecânica de escalada na estrutura da torre[cite: 14]

y -= velocidade_escalada;


// Efeito visual dos olhos emitindo sinais de rede instáveis[cite: 14]

timer_piscar_olhos -= 1;

```

```
if (timer_piscar_olhos <= 0) {  
    image_blend = choose(c_red, c_yellow, c_white); // Sinais instáveis  
    timer_piscar_olhos = irandom_range(5, 15);  
}
```

Interações de Cenário

```
// --- obj_chao_acido (Evento Step) ---  
  
// Gera partículas visuais de ácido evaporando o asfalto em furos fumegantes[cite:  
12]  
  
if (irandom(100) < 10) {  
    effect_create_above(ef_smoke, x + irandom_range(-16, 16), y, 0.5, c_green);  
}
```

```
// Interação de dano  
  
if (place_meeting(x, y, obj_david)) {  
    if (obj_david.escudo_ativo == false) {  
        // Causa dano contínuo se o escudo não foi ativado no totem[cite: 12]  
        obj_david.hp -= 1;  
    }  
}
```

Abismos Invisíveis e Plataformas de Código (Zona Urbana)

```
// --- obj_plataforma_invisivel (Evento Create) ---  
  
// Inicialmente, o vazio absoluto abre-se como um abismo[cite: 13]  
  
solid = false;  
  
image_alpha = 0; // Plataforma fantasmagórica invisível  
  
estado_css = "none";  
  
  
  
// --- obj_plataforma_invisivel (Evento de Colisão com obj_tiro_depuracao) ---  
  
// Quando David dispara o comando 'display: block;'[cite: 13]
```

```

if (other.comando_injetado == "display: block;"){

    estado_css = "block";

    solid = true; // Os caracteres transformam-se em blocos sólidos sob os pés[cite:
13]

    // Animação de materialização

    image_alpha = lerp(image_alpha, 1.0, 0.1);

    sprite_index = spr_bloco_html_brilhante; // HTML a brilhar em branco sobre o
fundo escuro[cite: 13]

}

```

1. Sistema do David e Gestão de RAM (Subsolo do Metrô)

```

// --- obj_david (Evento Create) ---

hp = 100;

velocidade_base = 5;

velocidade_atual = 5;

carga_ram_atual = 0;

carga_ram_maxima = 1024; // Medido em Megabytes simulados

escudo_ativo = false;

mostrar_alerta_hud = "";

// --- obj_david (Evento Step) ---

var _move = keyboard_check(vk_right) - keyboard_check(vk_left);

// Calcula a penalização de velocidade baseada na RAM ocupada

var _penalizacao_ram = (carga_ram_atual / carga_ram_maxima) * 3;

velocidade_atual = max(1, velocidade_base - _penalizacao_ram);

x += _move * velocidade_atual;

```

```

// Alerta de RAM crítica e otimização manual por descarte (Tecla Q)
if (carga_ram_atual > (carga_ram_maxima * 0.75)) {
    mostrar_alerta_hud = "RAM CRÍTICA: OTIMIZE ARMAZENAMENTO";
    if (keyboard_check_pressed(ord("Q"))) {
        carga_ram_atual = max(0, carga_ram_atual - 256); // Descarta pacotes
        superfluos[cite: 12]
        mostrar_alerta_hud = "";
    }
}

```

2. Boss 1: O Zumbi da Esteira (Loop Infinito)

```

// --- obj_boss_esteira (Evento Create) ---
loopCondition = true;
timer_clonagem = room_speed * 3;
velocidade_esteira = 10;

// --- obj_boss_esteira (Evento Step) ---
if (loopCondition == true) {
    timer_clonagem -= 1;
    if (timer_clonagem <= 0) {
        instance_create_layer(x, y - 64, "Inimigos", obj_clone_zumbi);
        timer_clonagem = room_speed * 3;
    }
} else {
    // A esteira engasga, chia e desacelera até travar[cite: 12]
    velocidade_esteira = lerp(velocidade_esteira, 0, 0.05);
    if (velocidade_esteira < 0.1) velocidade_esteira = 0;
}

```

```
// --- obj_caixa_energia (Evento de Colisão com obj_tiro_depuracao) ---  
// O tiro insere o comando 'break' na variável de controlo[cite: 12]  
obj_boss_esteira.loopCondition = false;  
instance_destroy();
```

3. Boss 2: O Sequestrador de IP

```
// --- obj_sequestrador_ip (Evento Step) ---  
if (estado == "ATACANDO" && irandom(100) < 5) {  
    var_ddos = instance_create_layer(x, y, "Projeteis", obj_pacote_ddos);  
    _ddos.image_blend = c_red;  
    _ddos.direction = point_direction(x, y, obj_david.x, obj_david.y);  
    _ddos.speed = 8;  
}
```

```
// --- obj_servidor_ip (Evento de Colisão com obj_tiro_depuracao) ---  
// Análise da máscara 255.255.255.192  
if (ip_valido == true) {  
    with (obj_sequestrador_ip) {  
        estado = "VULNERAVEL";  
        mensagem_sistema = "IP verdadeiro detectado!";[cite: 13]  
        alarm[0] = room_speed * 1.5; // Intervalo minúsculo para o ataque  
        concentrado[cite: 13]  
    }  
}
```

4. Zona Urbana HTML/CSS & Aranha Script Injector

```
// --- obj_plataforma_invisivel (Evento de Colisão com obj_tiro_depuracao) ---  
// Disparo do comando 'display: block;' para materializar plataformas[cite: 13]  
if (other.comando_injetado == "display: block;"){  
    solid = true;
```

```

image_alpha = 1.0;

sprite_index = spr_bloco_html_brilhante;

}

// --- obj_pilar_arena (Evento de Colisão com obj_tiro_depuracao) ---
// Reescrevendo as classes da Aranha Script Injector[cite: 13]
if (classe_alvo == "gravidade") {
    global.mundo_rotacao = 0; // "classe.gravidade { girar: 0deg; }"[cite: 13]
} else if (classe_alvo == "luz") {
    global.mundo_opacidade = 1.0; // "classe.luz { visibilidade: visivel; }"[cite: 13]
}

```

5. Instalação Científica e Boss: O Chefe Vértice

```

// --- obj_chefe_vertice (Evento Step) ---
x += velocidade_x;

var _x_grafico = (x - x_centro_sala) / escala_grafica;
var _y_grafico = (-power(_x_grafico, 2)) + (4 * _x_grafico);
y = y_base_chao - (_y_grafico * escala_grafica);

// --- obj_armadilha_energia (Evento Step) ---
// Posicionamento exato no vértice calculado (-b / 2a)[cite: 14]
var _vertice_x = x_centro_sala + (2 * escala_grafica);

if (abs(obj_chefe_vertice.x - _vertice_x) < 15) {
    instance_create_layer(x, y, "Efeitos", obj_clarao_branco);
    with (obj_chefe_vertice) {
        estado = "QUEDA_LIVRE";
        gravity = 0.5; // Desfaz-se como giz[cite: 14]
    }
}

```

```
}
```

6. Boss Final: Paul e o Terminal de Decisão

```
// --- obj_boss_paul (Evento de Colisão com obj_tiro_depuracao) ---
```

```
hp_armadura -= dano_depuracao;
```

```
if (hp_armadura <= 0 && estado != "DERROTADO") {
```

```
    estado = "DERROTADO";
```

```
    instance_create_layer(x, y, "Efeitos", obj_erro_fatal_vermelho);
```

```
    sprite_index = spr_paul_armadura_rompida; // Peças metálicas caem no  
    chão[cite: 15]
```

```
}
```

```
// --- obj_painel_final (Evento Step - Escolha de Finais) ---
```

```
if (mouse_check_button_pressed(mb_left)) {
```

```
    if (mouse_x >= x_botao_azul_esq && mouse_x <= x_botao_azul_dir) {
```

```
        global.final_escolhido = "CURA"; // Inject Cure();[cite: 15]
```

```
        room_goto(rm_cinematica_final_azul);
```

```
    } else if (mouse_x >= x_botao_vermelho_esq && mouse_x <=  
x_botao_vermelho_dir) {
```

```
        global.final_escolhido = "CONTROLO"; // Consolidate Control();[cite: 15]
```

```
        room_goto(rm_cinematica_final_vermelho);
```

```
    }
```

```
}
```

Gestão de Inventário e Módulos (Armadura e Blaster)

1. Estrutura de Dados do Inventário e Slots

```
// --- obj_inventario_manager (Evento Create) ---
```

```
// Slots da Armadura (Exoesqueleto)
```

```

armadura_slots = {
    modulo_ram: noone,    // Expande o limite máximo de RAM
    protetor_rede: noone, // Mitiga danos de ataques DDoS e ácido
    nucleo_energia: noone // Aumenta a eficiência dos Power Cores
};

// Slots do Blaster de Depuração
blaster_slots = {
    tipo_projetil: "Padrao", // Pode mudar para "Comentario", "TryCatch", etc.
    modulo_alcance: noone,    // Aumenta a velocidade e alcance do tiro
    modulo_burst: noone      // Permite disparos múltiplos de pacotes de dados
};

// Inventário geral de itens guardados em memória (RAM)
inventario_itens = [];
capacidade_maxima_mochila = 1024; // Megabytes

```

2. Interface de Upgrades da Armadura

```

// --- obj_ui_armadura (Evento de Interação / Tecla I) ---
// Função para instalar uma Gear na Armadura
function instalar_gear_armadura(_tipo_slot, _item_gear) {
    switch(_tipo_slot) {
        case "ram":
            armadura_slots.modulo_ram = _item_gear;

            // Aumenta a RAM máxima do David com base no upgrade
            obj_david.carga_ram_maxima = 1024 + _item_gear.bonus_capacidade;

            break;

        case "rede":

```

```

    armadura_slots.protetor_rede = _item_gear;

    // Reduz o dano recebido de ambientes corrosivos (ex: ácido verde)

    obj_david.resistencia_rede = _item_gear.fator_mitigacao;

    break;

}

audio_play_sound(snd_gear_equipada, 1, false);

}

```

3. Interface de Modificação do Blaster

```

// --- obj_ui_blaster (Evento de Modificação) ---

// Função para atualizar os módulos do Blaster de Depuração

function equipar_modulo_blaster(_slot, _modulo){

    if (_slot == "projatil"){

        blaster_slots.tipo_projatil = _modulo.nome; // Ex: "InjetorTryCatch"

        obj_blaster.dano = _modulo.dano_base;

        obj_blaster.cor_tiro = _modulo.cor_visual;

    } else if (_slot == "alcance"){

        blaster_slots.modulo_alcance = _modulo;

        obj_blaster.velocidade_tiro = 12 + _modulo.bonus_velocidade;

    }

}

```

4. Desenho da HUD de Inventário (Draw GUI)

```

// --- obj_controlador_inventario_gui (Evento Draw GUI) ---

if (global.menu_inventario_aberto){

    draw_set_color(c_black);

    draw_set_alpha(0.8);

    draw_rectangle(100, 100, 864, 568, false); // Fundo do painel do terminal

    draw_set_color(c_lime);

```

```

draw_set_font(fnt_retro_terminal);

draw_text(120, 120, "=== PAINEL DE CONFIGURAÇÃO DO SISTEMA ===");


// Desenhar slots da Armadura

draw_text(120, 160, "[ SLOTS DA ARMADURA ]");

draw_text(140, 190, "RAM Slot: " +
string(obj_inventario_manager.armadura_slots.modulo_ram));

draw_text(140, 220, "Rede Slot: " +
string(obj_inventario_manager.armadura_slots.protetor_rede));


// Desenhar slots do Blaster

draw_text(120, 280, "[ SLOTS DO BLASTER DE DEPURACÃO ]");

draw_text(140, 310, "Tipo de Pacote: " +
string(obj_inventario_manager.blaster_slots.tipo_projetil));

draw_text(140, 340, "Módulo Alcance: " +
string(obj_inventario_manager.blaster_slots.modulo_alcance));
}

1. Objeto do Terminal de Crafting

// --- Evento Create ---

ativo = false;

distancia_interacao = 40;


// Lista de Receitas Disponíveis para Crafting

receitas = [

    { nome: "Módulo RAM +256MB", custo_energia: 2, custo_dados: 50, tipo: "ram",
valor: 256 },

    { nome: "Protetor contra Ácido/DDoS", custo_energia: 3, custo_dados: 100, tipo:
"rede", valor: 0.5 },

    { nome: "Módulo Blaster Burst", custo_energia: 4, custo_dados: 150, tipo:
"blaster", valor: 2 }

```

```
];
```

```
selecionado = 0;
```

2. Lógica de Interação e Verificação de Recursos (Step Event)

```
// --- Evento Step ---
```

```
var _dist = distance_to_object(obj_david);
```

```
var _interagir = keyboard_check_pressed(ord("E"));
```

```
// Alterna o estado do terminal se o jogador estiver perto
```

```
if (_dist <= distancia_interacao && _interagir) {
```

```
    ativo = !ativo;
```

```
    audio_play_sound(snd_terminal_beep, 1, false);
```

```
}
```

```
// Se o terminal estiver aberto, permite navegar pelas receitas e fabricar
```

```
if (ativo) {
```

```
    // Navegar pelo menu com as setas para cima/baixo
```

```
    if (keyboard_check_pressed(vk_down)) {
```

```
        selecionado = (selecionado + 1) % array_length(receitas);
```

```
    }
```

```
    if (keyboard_check_pressed(vk_up)) {
```

```
        selecionado = (selecionado - 1 + array_length(receitas)) %  
array_length(receitas);
```

```
    }
```

```
// Pressionar ENTER para confirmar o Crafting
```

```
if (keyboard_check_pressed(vk_enter)) {
```

```

var _receita_atual = receitas[selecionado];

// Verifica se o jogador tem recursos suficientes (Energia Renovável e
Pontos/Dados)

if (global.energia_renovavel >= _receita_atual.custo_energia &&
global.pontuacao >= _receita_atual.custo_dados) {

    // Consome os recursos

    global.energia_renovavel -= _receita_atual.custo_energia;

    global.pontuacao -= _receita_atual.custo_dados;

    // Aplica o upgrade no inventário do David

    if (_receita_atual.tipo == "ram") {

        obj_david.carga_ram_maxima += _receita_atual.valor;

    } else if (_receita_atual.tipo == "rede") {

        obj_david.resistencia_rede += _receita_atual.valor;

    } else if (_receita_atual.tipo == "blaster") {

        obj_blaster.modos_burst = true;

    }

    audio_play_sound(snd_craft_sucesso, 1, false);

    show_debug_message("Crafting concluído: " + _receita_atual.nome);

} else {

    // Recursos insuficientes

    audio_play_sound(snd_erro_critico, 1, false);

}

}

}

```

3. Interface Visual do Terminal (Draw GUI Event)

```
// --- Evento Draw GUI ---

if (ativo) {

    var _x1 = 200;

    var _y1 = 150;

    var _x2 = 765;

    var _y2 = 450;


    // Fundo da janela do terminal holográfico

    draw_set_color(c_black);

    draw_set_alpha(0.85);

    draw_rectangle(_x1, _y1, _x2, _y2, false);


    draw_set_color(c_cyan);

    draw_set_alpha(1.0);

    draw_rectangle(_x1, _y1, _x2, _y2, true); // Borda do terminal


    // Textos do cabeçalho

    draw_set_font(fnt_retro_terminal);

    draw_text(_x1 + 20, _y1 + 20, "=== TERMINAL DE FABRICAÇÃO DE GEARS ===");

    draw_text(_x1 + 20, _y1 + 50, "Power Cores Disponíveis: " +
string(global.energia_renovavel));

    draw_text(_x1 + 20, _y1 + 70, "Dados (Score): " + string(global.pontuacao));


    // Listar Receitas

    for (var i = 0; i < array_length(receitas); i++) {

        var _r = receitas[i];

        var _pos_y = _y1 + 110 + (i * 40);
```

```

// Destaca a linha selecionada

if (i == selecionado) {

    draw_set_color(c_lime);

    draw_text(_x1 + 20, _pos_y, ">" + _r.nome + " [Custo: " +
string(_r.custo_energia) + " Cores | " + string(_r.custo_dados) + " Dados]");

} else {

    draw_set_color(c_white);

    draw_text(_x1 + 40, _pos_y, _r.nome + " [Custo: " + string(_r.custo_energia) + "
Cores | " + string(_r.custo_dados) + " Dados]");

}

}

```

```

// Instruções de rodapé

draw_set_color(c_yellow);

draw_text(_x1 + 20, _y2 - 40, "Use [SETAS] para navegar e [ENTER] para fabricar.");

}

```

1. Sistema de Gestão de Fases e Transição (obj_fase_manager)

```

// --- Evento Create ---

// Define as fases com base no roteiro oficial

fase_atual = 1;

nome_fase = "Zona Urbana Desvastada";


// Dicionário ou enumeração das Rooms correspondentes no projeto

sala_atual_room = rm_zona_urbana;


function avançar_fase() {

    fase_atual += 1;

```

```

switch(fase_atual) {
    case 2:
        nome_fase = "Subsolo do Metrô";
        sala_atual_room = rm_subsolo_metro;
        break;
    case 3:
        nome_fase = "Distrito HTML/CSS";
        sala_atual_room = rm_zona_html_css;
        break;
    case 4:
        nome_fase = "Instalação Científica (Vértice)";
        sala_atual_room = rm_instalacao_cientifica;
        break;
    case 5:
        nome_fase = "Torre de Transmissão";
        sala_atual_room = rm_torre_transmissao;
        break;
    case 6:
        nome_fase = "Núcleo do Servidor Central (Boss Final)";
        sala_atual_room = rm_nucleo_servidor;
        break;
}

// Dispara a transição visual e muda de sala
room_goto(sala_atual_room);
audio_play_sound(snd_transicao_fase, 1, false);
}

```

2. O Portal de Conclusão de Nível (obj_portal_transicao)

```
// --- Evento de Colisão com obj_david ---

// Verifica se o jogador concluiu o objetivo principal da fase (ex: derrotar o boss ou
ativar portões lógicos)

if (global.objetivo_fase_concluido == true) {

    // Inicia a transição de fases

    if (instance_exists(obj_fase_manager)) {

        obj_fase_manager.avançar_fase();

    }

}
```

3. Ecrã de Transmissão / Carregamento entre Fases (Draw GUI Event)

```
// --- obj_transicao_tela (Evento Draw GUI) ---

// Executado numa sala dedicada a transições (ex: rm_loading)

draw_set_color(c_black);

draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);


draw_set_color(c_lime);

draw_set_font(fnt_retro_terminal);

draw_text(100, 100, ">>> A CARREGAR PROTOCOLO...");

draw_text(100, 140, "FASE ATUAL: " + string(obj_fase_manager.nome_fase));

draw_text(100, 180, "ESTADO DA RAM: " + string(obj_david.carga_ram_atual) + "MB
/ " + string(obj_david.carga_ram_maxima) + "MB");


// Temporizador automático para passar para o gameplay da fase após 3 segundos

if (!alarm[0]) {

    alarm[0] = room_speed * 3;

}


// --- Evento Alarm 0 ---
```

```
room_goto(obj_fase_manager.sala_atual_room);
```

1. Transição da Fase 1 para a Fase 2 (Zona Urbana → Subsolo do Metrô)

```
// --- obj_transicao_fase_1_2 (Evento Step) ---
```

```
// Efeito de queda de dados verticais antes de mudar de sala
```

```
if (ativo) {
```

```
    alfa_fade += 0.05;
```

```
    if (alfa_fade >= 1.0) {
```

```
        room_goto(rm_subsolo_metro); // Entra no subsolo com cabos de fibra  
óptica[cite: 12]
```

```
    }
```

```
}
```

```
// --- Evento Draw GUI ---
```

```
draw_set_color(c_black);
```

```
draw_set_alpha(alfa_fade);
```

```
draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);
```

```
// Desenha linhas de fibra óptica a piscar na tela
```

```
draw_set_color(c_red);
```

```
draw_text(100, 100, ">>> A DESCENDER AO SUBSOLO DO METRÔ...");
```

```
draw_text(100, 130, ">>> FIBRA ÓPTICA DETETADA NOS TRILHOS[cite: 12].");
```

2. Transição da Fase 2 para a Fase 3 (Subsolo do Metrô → Zona Urbana HTML/CSS)

```
// --- obj_transicao_fase_2_3 (Evento Create) ---
```

```
tempo_carregamento = 0;
```

```
texto_carregamento = "Carregando estilo... display: block;";
```

```
// --- Evento Step ---
```

```
tempo_carregamento += 1;
```

```

if (tempo_carregamento > room_speed * 2) {
    room_goto(rm_zona_html_css); // Entra na zona web[cite: 13]
}

// --- Evento Draw GUI ---
draw_set_color(c_black);
draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);

draw_set_color(c_white);
draw_set_font(fnt_retro_terminal);
draw_text(150, 200, "<div> Reestruturando Distrito Web... </div>");
draw_text(150, 240, texto_carregamento);

```

3. Transição da Fase 3 para a Fase 4 (Zona Urbana HTML/CSS → Instalação Científica)

```

// --- obj_transicao_fase_3_4 (Evento Step) ---
escala_grade += 0.1;
if (escala_grade >= 5.0) {
    room_goto(rm_instalacao_cientifica); // Entra na instalação matemática
}

// --- Evento Draw GUI ---
draw_set_color(c_navy);
draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);

// Desenha eixos cartesianos virtuais na transição
draw_set_color(c_aqua);
draw_line(0, display_get_gui_height()/2, display_get_gui_width(),
display_get_gui_height()/2);

```

```
draw_line(display_get_gui_width()/2, 0, display_get_gui_width()/2,
display_get_gui_height());

draw_text(200, 150, "Sincronizando Eixos:  $y = ax^2 + bx + c$ [cite: 14]");
```

4. Transição da Fase 4 para a Fase 5 (Instalação Científica → Torre de Transmissão)

```
// --- obj_transicao_fase_4_5 (Evento Step) ---

timer_relampago--;

if (timer_relampago <= 0) {

    audio_play_sound(snd_trovao_digital, 1, false);

    room_goto(rm_torre_transmissao); // Sobe para o topo da torre[cite: 14]

}


// --- Evento Draw GUI ---

// Pisca luz branca simulando relâmpagos de dados

draw_set_color(choose(c_white, c_black));

draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);


draw_set_color(c_yellow);

draw_text(100, 100, "ATENÇÃO: A escalando a Torre de Transmissão...[cite: 14]");
```

5. Transição da Fase 5 para a Fase 6 (Torre de Transmissão → Núcleo do Servidor Central / Boss Final)

```
// --- obj_transicao_fase_5_6 (Evento Step) ---

opacidade_nucleo += 0.02;

if (opacidade_nucleo >= 1.0) {

    room_goto(rm_nucleo_servidor); // Confronto final contra Paul[cite: 15]

}


// --- Evento Draw GUI ---
```

```
draw_set_color(c_blue);

draw_set_alpha(opacidade_nucleo);

draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);


draw_set_color(c_white);

draw_set_alpha(1.0);

draw_text(200, 200, "CONEXÃO ESTABELECIDA COM O NÚCLEO CENTRAL.");

draw_text(200, 240, "Aviso: Presença de Administrador detetada (Paul)[cite: 15].");
```

1. Animação de Morte de David (Desfragmentação de Dados)

```
// --- obj_david (Evento Step - Chegada a 0 de HP) ---

if (hp <= 0 && estado != "MORTO") {

    estado = "MORTO";

    sprite_index = spr_david_morte_glitch; // Animação de desintegração em pixels

    image_speed = 1.0;


    // Efeito sonoro de erro crítico de sistema

    audio_play_sound(snd_erro_critico, 1, false);


    // Desativa controlos de movimento

    hsp = 0;

    vsp = 0;


    // Inicia temporizador para o ecrã de Game Over / Reiniciar

    alarm[1] = room_speed * 2.5;

}
```

```
// --- obj_david (Evento Alarm 1 - Game Over) ---

// Reinicia o nível ou regressa ao último checkpoint de salvamento de rede
```

```
room_restart();
```

2. Animação de Morte dos Cyber-Zombies e Clones

```
// --- obj_cyber_zombie / obj_clone_zumbi (Evento Create) ---
```

```
morto = false;
```

```
// --- obj_cyber_zombie (No código de receber dano do Blaster) ---
```

```
hp -= dano_recebido;
```

```
if (hp <= 0 && !morto) {
```

```
    morto = true;
```

```
    sprite_index = spr_zombie_death; // Animação de queda do ciborgue[cite: 10]
```

```
    image_index = 0;
```

```
    image_speed = 1.0;
```

```
    hsp = 0;
```

```
}
```

```
// --- obj_cyber_zombie (Evento Animation End) ---
```

```
if (morto) {
```

```
    // Quando a animação de queda termina, transita para o efeito de caveira com  
    glitch[cite: 10]
```

```
    if (sprite_index == spr_zombie_death) {
```

```
        sprite_index = spr_glitch_skull;[cite: 10]
```

```
        alarm[0] = room_speed * 1.5; // Tempo para o holograma desvanecer
```

```
    }
```

```
}
```

```
// --- obj_cyber_zombie (Evento Alarm 0) ---
```

```
// Limpa o objeto da RAM do jogo para otimizar desempenho
```

```
instance_destroy();
```

3. Animação de Morte do Zumbi da Esteira (Boss 1)

```
// --- obj_boss_esteira (Evento Step - Derrota) ---
```

```
if (hp_boss <= 0 && estado != "DESTRUIDO") {
```

```
    estado = "DESTRUIDO";
```

```
    sprite_index = spr_boss_esteira_colapso;
```

```
    // Efeito de explosões em cadeia de pixéis
```

```
    repeat(5) {
```

```
        instance_create_layer(x + irandom_range(-50, 50), y + irandom_range(-50, 50),  
"Efeitos", obj_explosao_pixel);
```

```
    }
```

```
    audio_play_sound(snd_sistema_abaixo, 1, false);
```

```
    // Liberta o caminho e abre o portal para o subsolo
```

```
    global.objetivo_fase_concluido = true;
```

```
    instance_create_layer(x, y, "Instances", obj_portal_transicao);
```

```
    // Desativa o boss após a animação
```

```
    alarm[2] = room_speed * 3;
```

```
}
```

```
// --- obj_boss_esteira (Evento Alarm 2) ---
```

```
instance_destroy();
```

4. Animação de Morte do Sequestrador de IP (Boss 2)

```
// --- obj_sequestrador_ip (Evento Step - Derrota) ---
```

```

if (hp <= 0 && estado != "MORTO") {
    estado = "MORTO";
    sprite_index = spr_sequestrador_desfragmentando;

    // O corpo desfaz-se em múltiplos pacotes de dados corrompidos
    for (var i = 0; i < 8; i++) {
        var _fragmento = instance_create_layer(x, y, "Efeitos",
obj_pacote_erro_disperso);
        _fragmento.direction = i * 45;
        _fragmento.speed = 4;
    }

    audio_play_sound(snd_curto_circuito_maciço, 1, false);
    alarm[0] = room_speed * 2; // Desaparece após dispersar os dados
}

// --- obj_sequestrador_ip (Evento Alarm 0) ---
instance_destroy();

```

5. Animação de Morte da Aranha Script Injector (Boss 3)

```

// --- obj_aranha_css (Evento Step - Derrota) ---
if (hp <= 0 && estado != "QUEDA_FINAL") {
    estado = "QUEDA_FINAL";
    sprite_index = spr_aranha_derrotada;

    // Efeito de colapso das linhas de código CSS/JavaScript
    vsp = 4; // Queda pesada no centro da arena[cite: 13]
    y += vsp;

```

```

if (place_meeting(x, y + 1, obj_chao)) {

    // Impacto final no chão que gera uma onda de choque de código

    instance_create_layer(x, y, "Efeitos", obj_onda_choque_css);

    audio_play_sound(snd_impacto_pesado, 1, false);


    // Conclui a fase e liberta o acesso científico

    global.objetivo_fase_concluido = true;

    instance_create_layer(x, y, "Instances", obj_portal_transicao);


    instance_destroy();
}
}

```

6. Animação de Morte do Chefe Vértice (Boss 4)

```

// --- obj_chefe_vertice (Evento Step - Derrota) ---

if (hp <= 0 && estado != "DESMANCHE") {

    estado = "DESMANCHE";

    sprite_index = spr_vertice_desfazendo;


    // Os números e fórmulas matemáticas ao redor começam a cair em queda livre

    gravity = 0.8;

    vsp += gravity;

    y += vsp;


    // Cria partículas em forma de texto matemático a dissipar-se

    if (irandom(100) < 30) {

```

```

        instance_create_layer(x + irandom_range(-30, 30), y, "Efeitos",
obj_particula_equacao);
    }

    // Ao atingir o plano gráfico inferior, desvanece por completo[cite: 14]
    if (place_meeting(x, y, obj_chao)) {
        audio_play_sound(snd_silencio_digital, 1, false);
        global.objetivo_fase_concluido = true;
        instance_create_layer(x, y, "Instances", obj_portal_transicao);
        instance_destroy();
    }
}

```

1. Movimento de Nuvens no Céu (Efeito Paralaxe)

```

// --- obj_gerenciarador_ceu (Evento Create) ---

// Obtém o ID da camada de fundo onde estão as nuvens
camada_nuvens = layer_get_id("Background_Nuvens");
velocidade_nuvens = 0.8; // Velocidade de deslocamento horizontal

// --- obj_gerenciarador_ceu (Evento Step) ---

// Move a camada de nuvens continuamente a cada frame
if (layer_exists(camada_nuvens)) {
    layer_x(camada_nuvens, layer_get_x(camada_nuvens) - velocidade_nuvens);
}

```

2. Timer de Fase: Chuva Ácida e Resistência dos Escudos

```

// --- obj_acid_rain_timer (Evento Create) ---

tempo_limite_fase = 180; // 180 segundos (3 minutos) para concluir a fase

```

```
tempo_atual = tempo_limite_fase;
```

```
resistencia_escudo = 100; // Durabilidade máxima do escudo hexagonal de David
```

```
taxa_corrosao = 2; // Velocidade com que o ácido corrói o escudo/vida
```

```
// --- obj_acid_rain_timer (Evento Step) ---
```

```
if (tempo_atual > 0) {
```

```
    // Subtrai o tempo real usando o delta_time (convertido para segundos)
```

```
    tempo_atual -= delta_time / 1000000;
```

```
} else {
```

```
    // Tempo esgotado: Falha crítica por corrupção ambiental
```

```
    room_restart();
```

```
}
```

```
// Se a chuva ácida estiver ativa no nível atual
```

```
if (global.chuva_acido_ativa == true) {
```

```
    if (obj_david.escudo_ativo == true) {
```

```
        // O escudo protege David, mas perde resistência gradualmente
```

```
        resistencia_escudo -= 0.05;
```

```
        if (resistencia_escudo <= 0) {
```

```
            obj_david.escudo_ativo = false; // O escudo colapsa por sobrecarga ácida
```

```
            audio_play_sound(snd_escudo_quebrado, 1, false);
```

```
        }
```

```
    } else {
```

```
        // Sem escudo ativo, o ácido causa dano direto à vida de David
```

```
        obj_david.hp -= taxa_corrosao * (1 / room_speed);
```

```
    }
```

```
}
```

3. Transição de Dia para Noite com o Decorrer das Fases

```
// --- obj_transicao_dia_noite (Evento Create) ---
```

```
// Lê a fase atual gerida pelo gestor global do jogo
```

```
fase_atual = obj_fase_manager.fase_atual;
```

```
// Define a cor base do céu e a opacidade da noite de acordo com a progressão das fases
```

```
switch(fase_atual) {
```

```
    case 1: // Zona Urbana (Entardecer corrompido)
```

```
        cor_ceu = make_color_rgb(220, 120, 80);
```

```
        alpha_noite = 0.25;
```

```
        break;
```

```
    case 2: // Subsolo do Metrô (Cinza industrial)
```

```
        cor_ceu = make_color_rgb(80, 100, 120);
```

```
        alpha_noite = 0.45;
```

```
        break;
```

```
    case 3: // Distrito HTML/CSS (Roxo digital)
```

```
        cor_ceu = make_color_rgb(60, 30, 100);
```

```
        alpha_noite = 0.60;
```

```
        break;
```

```
    case 4: // Instalação Científica (Azul néon)
```

```
        cor_ceu = make_color_rgb(20, 40, 90);
```

```
        alpha_noite = 0.75;
```

```
        break;
```

```
    case 5: // Torre de Transmissão (Noite de tempestade digital)
```

```
        cor_ceu = make_color_rgb(10, 15, 40);
```

```

    alpha_noite = 0.85;

    break;

case 6: // Núcleo do Servidor Central (Escuridão profunda / Noite total)

    cor_ceu = make_color_rgb(2, 2, 10);

    alpha_noite = 0.95;

    break;

default:

    cor_ceu = c_black;

    alpha_noite = 0.5;

    break;

}

// --- obj_transicao_dia_noite (Evento Draw GUI) ---

// Desenha uma sobreposição em toda a tela para tingir o ambiente com a
iluminação da fase

draw_set_color(cor_ceu);

draw_set_alpha(alpha_noite);

draw_rectangle(0, 0, display_get_gui_width(), display_get_gui_height(), false);

// Reseta o alpha para evitar interferências em outras HUDs

draw_set_alpha(1.0);

```

1. Estrutura de Dados das Skins (obj_skin_manager)

```

// --- Evento Create ---

// Registo de todas as skins colecionáveis baseadas na lore do jogo

skins_disponiveis = [

    { id: "default", nome: "Padrão (Explorador Órbita)", desbloqueada: true,
cor_viseira: c_violet, bônus: "Nenhum" },

```

```

    { id: "neon_css", nome: "Desenvolvedor CSS Glow", desbloqueada: false,
cor_viseira: c_cyan, bônus: "+10% Velocidade de Movimento" },

    { id: "carbon_paul", nome: "Protetor Criptografado", desbloqueada: false,
cor_viseira: c_red, bônus: "+20% Resistência a Dano" },

    { id: "matrix_glitch", nome: "Glitch 9 Nômade", desbloqueada: false, cor_viseira:
c_lime, bônus: "Invisibilidade parcial a Cyber-Zombies" }

];

```

```

skin_atual_index = 0; // Começa com a skin padrão

```

2. Lógica de Coleta de Skins pelo Cenário (obj_item_skin)

```

// --- Evento de Colisão com obj_david ---

var _id_skin_encontrada = "neon_css"; // Exemplo de skin escondida no nível

// Procura na lista e desbloqueia a skin correspondente
with (obj_skin_manager) {
    for (var i = 0; i < array_length(skins_disponiveis); i++) {
        if (skins_disponiveis[i].id == _id_skin_encontrada) {
            if (!skins_disponiveis[i].desbloqueada) {
                skins_disponiveis[i].desbloqueada = true;
                audio_play_sound(snd_skin_desbloqueada, 1, false);
                show_debug_message("Nova Skin Desbloqueada: " +
skins_disponiveis[i].nome);
            }
            break;
        }
    }
}
}

```

```
// Destroi o item coletável do mapa  
instance_destroy();
```

3. Interface de Personalização e Troca de Skins (Draw GUI Event)

```
// --- obj_skin_manager (Evento Draw GUI - Se o menu de skins estiver aberto) if  
(global.menu_skins_aberto) {  
  
    var _x1 = 250;  
  
    var _y1 = 120;  
  
    var _x2 = 780;  
  
    var _y2 = 480;  
  
  
    // Fundo da janela de personalização  
    draw_set_color(c_black);  
  
    draw_set_alpha(0.9);  
  
    draw_rectangle(_x1, _y1, _x2, _y2, false);  
  
  
    draw_set_color(c_fuchsia);  
  
    draw_rectangle(_x1, _y1, _x2, _y2, true);  
  
  
    draw_set_font(fnt_retro_terminal);  
  
    draw_text(_x1 + 20, _y1 + 20, "=== VESTUÁRIO E SKINS DE DAVID ===");  
  
  
    // Listar as skins disponíveis para escolha  
    for (var i = 0; i < array_length(skins_disponiveis); i++) {  
  
        var _s = skins_disponiveis[i];  
  
        var _pos_y = _y1 + 70 + (i * 45);
```

```

var _status = _s.desbloqueada ? "[DESBLOQUEADA]" : "[BLOQUEADA]";

if (i == skin_atual_index) {
    draw_set_color(c_lime);

    draw_text(_x1 + 20, _pos_y, ">" + _s.nome + " " + _status);

    draw_text(_x1 + 40, _pos_y + 20, " Bônus: " + _s.bônus);
} else {
    draw_set_color(c_white);

    draw_text(_x1 + 40, _pos_y, _s.nome + " " + _status);
}
}

draw_set_color(c_yellow);

draw_text(_x1 + 20, _y2 - 35, "Use [SETAS] para navegar e [ENTER] para equipar.");
}

```

4. Aplicação Dinâmica da Skin no David (Step Event)

```

// --- obj_skin_manager (Evento Step para alteração ativa) ---

if (global.menu_skins_aberto) {
    if (keyboard_check_pressed(vk_down)) {
        skin_atual_index = (skin_atual_index + 1) % array_length(skins_disponiveis);
    }

    if (keyboard_check_pressed(vk_up)) {
        skin_atual_index = (skin_atual_index - 1 + array_length(skins_disponiveis)) %
array_length(skins_disponiveis);
    }

    if (keyboard_check_pressed(vk_enter)) {

```

```
var _selecionada = skins_disponiveis[skin_atual_index];

if (_selecionada.desbloqueada) {
    // Aplica os sprites do David com base na skin escolhida
    switch(_selecionada.id) {
        object_set_sprite(obj_david, spr_david_idle_neon);
        obj_david.bonus_velocidade_ativo = 1.1;
        break;
        case "carbon_paul":
            object_set_sprite(obj_david, spr_david_idle_carbon);
            obj_david.resistencia_dano = 0.8;
            break;
        default:
            object_set_sprite(obj_david, spr_david_idle);
            break;
    }
    audio_play_sound(snd_gear Equipada, 1, false);
}
}
```